

Assignment 2: Coding Basics

Evan Lim

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.  
  
fivefive_sequence <- seq(1, 55, 5) # Creating a sequence from one to 55  
#increasing by fives and assigning name fivefive_sequence
```

```
#2.  
  
mean(fivefive_sequence) #Calculating the mean of the sequence
```

```
## [1] 26
```

```
"The mean of the sequence is 26"
```

```
## [1] "The mean of the sequence is 26"
```

```
median(fivefive_sequence) #Calculating the median of the sequence
```

```
## [1] 26
```

```
"The median is 26"
```

```
## [1] "The median is 26"
```

```
#3.
```

```
meanfivefive <- mean(fivefive_sequence) #Assigning a name to the mean
```

```
medianfivefive <- median(fivefive_sequence) #Assigning a name to the median
```

```
meanfivefive > medianfivefive # Checking if the mean is greater than the median,
```

```
## [1] FALSE
```

```
#output is FALSE
```

```
medianfivefive > meanfivefive # Checking if the median is greater than the mean,
```

```
## [1] FALSE
```

```
#output is FALSE
```

```
meanfivefive == medianfivefive # Checking if the mean is equal to the median,
```

```
## [1] TRUE
```

```
#output is TRUE
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#5,6.
```

```
student_name <- c("John", "Luana", "Mishka", "Kelly") # Vector type: character
```

```
test_score <- c(75, 80, 87, 100) # Vector type: numeric
```

```
on_scholarship <- c(TRUE, TRUE, FALSE, FALSE) # Vector type: logical
```

```
#7.
```

```
student_info <- data.frame(student_name, test_score, on_scholarship) # Creating a  
#data frame from the vectors
```

```
#8.
```

```
names(student_info) <- c("Name", "Test result", "Scholarship Status") # Labeling  
#the columns of the data frame
```

9. QUESTION: How is this data frame different from a matrix?

Answer: While both data frames and matrices are composed of several vectors, data frames can host vectors that differ in the kinds of variables they contain. This data frame has numerical, character, and logical vectors, while a matrix could have only numerical or only character or only logical vectors.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word “Pass”; otherwise print the word “Fail”.
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

#10. Create a function using if...else

```
# Creating a function using if, else
function_one <- function(x) {
  if(x>50){
    print("Pass")}
  else{
    print("Fail")}
}
```

#11. Create a function using ifelse()

```
#Creating a function using ifelse
function_two <- function(x){
  ifelse(x>50, print("Pass"), print("Fail"))
}
```

#12a. Run the first function with the value 52.5

```
function_one(52.5) # Running function one, which prints "Pass"
```

```
## [1] "Pass"
```

#12b. Run the second function with the value 52.5

```
function_two(52.5) # Running function two, which prints "Pass"
```

```
## [1] "Pass"
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores  
# function_one(test_score) Running this function but commenting it out to enable  
#knitting
```

```
#13b. Run the second function with the vector of test scores  
function_two(test_score) # This works, printing "Pass" for all values (which are
```

```
## [1] "Pass"
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

```
#indeed greater than 50)
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: The `ifelse` option worked properly, while the `if...else` option resulted in an error. From my research on the web, I learned that many functions in R are vectorized, meaning they are capable of operating on all elements of a vector, as opposed to one at a time. It seems that ‘`if...else`’ is NOT vectorized, which is likely why when given a vector as an input, as opposed to just a singular value, it results in an error.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)